

数据结构

NOIP范围内，基本数据结构包括：

- (1) 有序表
- (2) 链表
- (3) 栈(普通栈，单调栈)
- (4) 队列(普通队列，单调队列，优先队列，双端队列)
- (5) hash表
- (6) ST表

有序表

NOIP历届真题中，裸的有序表考得不是很多，比如
NOIP2004合并果子，NOIP2016蚯蚓

题目描述

在一个果园里，多多已经将所有的果子打了下来，而且按果子的不同种类分成了不同的堆。多多决定把所有的果子合成一堆。

每一次合并，多多可以把两堆果子合并到一起，消耗的体力等于两堆果子的重量之和。可以看出，所有的果子经过 $n - 1$ 次合并之后，就只剩下一堆了。多多在合并果子时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为 1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

例如有 3 种果子，数目依次为 1，2，9。可以先将 1、2 堆合并，新堆数目为 3，耗费体力为 3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为 12，耗费体力为 12。所以多多总共耗费体力 $= 3 + 12 = 15$ 。可以证明 15 为最小的体力耗费值。

合并果子

通过题意找规律我们可以很容易发现，我们每次只需要找到最小的两堆果子合并，这样最终的代价一定最小，但是怎么找最小的两堆呢？每次合并之后把新的果子插入时间复杂度为 $O(n)$ 的，很慢，我们可以考虑优先队列。但是我们忽略了果子的有序性，比如后合并的果子一定比先合并的果子数量多，我们完全可以对原果子建一个有序表，合并的果子建一个有序表。然后每次取两个表中最小的两个就可以了，时间复杂度为 $O(N)$ 的，效率高于优先队列。

蚯蚓

本题中，我们将用符号 $\lfloor c \rfloor$ 表示对 c 向下取整，例如： $\lfloor 3.0 \rfloor = \lfloor 3.1 \rfloor = \lfloor 3.9 \rfloor = 3$ 。

蛐蛐国最近蚯蚓成灾了！隔壁跳蚤国的跳蚤也拿蚯蚓们没办法，蛐蛐国王只好去请神刀手来帮他们消灭蚯蚓。

蛐蛐国里现在共有 n 只蚯蚓（ n 为正整数）。每只蚯蚓拥有长度，我们设第 i 只蚯蚓的长度为 a_i ($i = 1, 2, \dots, n$)，并保证所有的长度都是非负整数（即：可能存在长度为 0 的蚯蚓）。

每一秒，神刀手会在所有的蚯蚓中，准确地找到最长的那一只（如有多个则任选一个）将其切成两半。神刀手切开蚯蚓的位置由常数 p （是满足 $0 < p < 1$ 的有理数）决定，设这只蚯蚓长度为 x ，神刀手会将其切成两只长度分别为 $\lfloor px \rfloor$ 和 $x - \lfloor px \rfloor$ 的蚯蚓。特殊地，如果这两个数的其中一个等于 0，则这个长度为 0 的蚯蚓也会被保留。此外，除了刚刚产生的两只新蚯蚓，其余蚯蚓的长度都会增加 q （是一个非负整数常数）。

蛐蛐国王知道这样不是长久之计，因为蚯蚓不仅会越来越多，还会越来越长。蛐蛐国王决定求助于一位有着洪荒之力的神秘人物，但是救兵还需要 m 秒才能到来……（ m 为非负整数）

蛐蛐国王希望知道这 m 秒内的战况。具体来说，他希望知道：

- m 秒内，每一秒被切断的蚯蚓被切断前的长度（有 m 个数）；
- m 秒后，所有蚯蚓的长度（有 $n + m$ 个数）。

蛐蛐国王当然知道怎么做啦！但是他想考考你……

蚯蚓

首先，我们需要想到如何快速找到最长的那一条蚯蚓，第一种办法就是用优先队列，但是有一个问题是，每隔一秒未被分割的蚯蚓都会变长 q 。虽然队列里面的蚯蚓相对长度还是有序的，但是由于不停的切蚯蚓，新加进去的蚯蚓的长度和队列中的蚯蚓的长度相对长度是不一样的，所以需要处理一下，因为队列中蚯蚓很多，而每次只是放进去两条蚯蚓，所以我们可以先让放进去的蚯蚓的相对长度和队列中的蚯蚓的相对长度一致。比如我们只需要保证队列中的蚯蚓都是原始长度就可以了，而真实长度也很好算，第几次切就代表生长时间，再乘以每秒长多少。

蚯蚓

但是该题优先队列只有65分，因为数据太大，操作次数太多，所以需要有一个线性的维护。

和合并果子一样，先被切的蚯蚓切成的两段一定比后被切的蚯蚓切成的两端长，因为他们都在同时长，并且先切最长的。

所以我们只需要维护三个有序表就可以了，分表表示蚯蚓原长，切割后第一段长，切割后第二段长，然后每次从三个数组中选出最长的切就可以了。

该题有一点细节，因为蚯蚓长度是相对长度，所以可以是负数，取数的时候一定要判断该数组是否还有蚯蚓，不然容易出问题



栈的应用

NOIP范围内，栈的应用主要有三种

- (1) 栈模拟
 - (2) 单调栈优化
 - (3) 表达式求值
- 



栈的应用

NOIP范围内，栈的应用主要有三种

- (1) 栈模拟
- (2) 单调栈优化
- (3) 表达式求值



小 W 的爱情密码

题目描述

小 W 终于鼓起勇气向小 M 表白，然而只是有勇气写情书。为了防止情书内容被同学窃取，小 W 给情书加密。小 M 的解密方式很简单，假设情书是字符串 S1，小 W 给她的解密串是 S2，小 M 会重复地完成“在 S1 中找到子串 S2 并删除”这一操作直到在 S1 中找不到 S2。假如你是小 M，请你确定情书的最终内容。（从前往后遍历的时候找到一个可以删除的立即删除，删除之后，继续遍历新的数组，继续删除）

输入

第一行一个字符串 S1

第二行一个字符串 S2

输出

输出只有一行，一个字符串 S，表示最终内容



小 W 的爱情密码

一道裸的栈模拟的题，记录每一个位置可以匹配的长度，当删除一个字符串之后，从未删的最后一个字符匹配的位置继续匹配。



数列操作

题目描述

先给定一个长度为 n 数列，再给定 m 个操作，现在需要维护五个操作：

- 1 x ：在光标的前面插入一个数字 x 。
- 2：删除光标前的最后一个数字，如果光标前没有数字则忽略。
- 3：左移一格光标，如果光标已经在最左边则忽略。
- 4：右移一格光标，如果光标已经在最右边则忽略。
- 5 k ：求前 k 个数的前缀和，保证 $k \leq$ 数列长度。

初始时光标在数列的最后一个数后面

数据范围： $n, m, k \leq 100000, x \leq \text{int}$

数列操作

该题看上去涉及到数列的插入和删除，如果要在有限的时间内完成，需要用到高级数据结构，但是实际上不需要，因为每次操作都是在光标处做操作，并且光标的移动每次只移动一格，所以我们可以把光标前后分别看做两段。

对于操作一、二，我们就看做前一个数组末尾做插入删除操作，时间复杂度为 $O(1)$ 。

对于操作三、四、我们可以看做前一个数组末尾减小/增加，后一个数组相应增大/减小，为了方便，我们都把光标处看做末尾，即第二个数组反过来看。

对于操作五，如果 k 在光标左边，那么直接求前缀和，如果在光标右边，那么就是左边数组之和+右边数组前缀和之差。

该题相当于维护两个对顶栈。

列车调度

已知有 n 列火车按照 $1, 2, 3, \dots, n$ 的顺序排列，中间有一个火车中转站，每辆火车先进入中转站再出栈，每次只能是最前面一辆车入中转站或者中转站最上面一辆车出站，现给定一个火车的出栈顺序，判断是否可能

输入：

5

4 3 5 2 1

输出：

YES

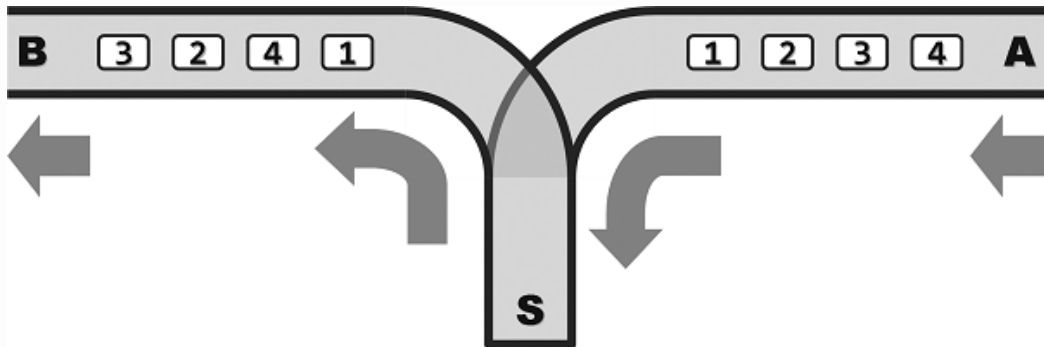
输入：

5

5 1 2 3 4

输出：

NO



列车调度

入栈的顺序肯定是 $1, 2 \dots n$ ，根据出栈的顺序，第一个出栈的数为 $a[1]$ ，那么 $1 \sim a[1]$ 之间数肯定都入栈了。我们依次把 $\leq a[1]$ 的数放入栈中，如果放完之后栈顶元素和 $a[1]$ 相等，那么栈顶元素出栈，此时栈内就不存在这个元素了；如果不相等，那么就说明这个序列错误。

比如3 1 2

第一个出栈的元素是3，那么1,2,3分别入栈，栈顶元素=3，接下来出栈的元素是1，而此时入栈的元素已经到了3，那么没有元素入栈，而此时栈顶元素不等于1，所以出栈顺序就是错误的。

单调栈优化

单调栈经常用来优化某些算法：

(1) 对于每一个数，寻找他右边第一个比他大(小)的数

(2) 对于给定序列，寻找一子序列最小(大)值*序列长度(之和)之积最大(小)

(3) 维护并查询序列的单调性

单调栈模板1

```
int top=0;//栈为空
for(int i=1;i<=n;i++)
{
    while(top!=0&& s[top]<a[i])
        top--; //循环执行出栈操作
    top++; //入栈
    s[top]=a[i];
}
```

因为对于当前这个数 $a[i]$ ，无论和栈顶元素关系如何，他都会入栈，所以我们可以先执行出栈操作，再执行入栈操作，这样就省掉了条件语句。

单调栈模板2

```
int top=0;
for(int i=1;i<=n;i++)
{
    int l=i;//如果没有出栈操作，左端点就为i
    while(top!=0&& s[top].num>a[i])
    {
        l=s[top].left;
        s[top].right=i-1;//出栈确定右端点
        top--;
        ...//计算面积
    }
    top++;
    s[top].left=l;//最后一个出栈元素的左端点
    //入栈确定左端点，但不一定是i
}
```

最长不下降子序列

已知一个长度为 n 的数列，求数列中的最长不下降子序列的长度。在序列中，对于所有的 $i < j$, $h[i] \leq h[j]$ 。

输入：

7

1 3 6 8 4 5 7

输出：

5

数据范围： $n \leq 100000$

最长不下降子序列是1 3 4 5 7，长度为5

最长不下降子序列

对于 $n=7$; 1 3 6 8 4 5 7

如果先不考虑数据范围，用常规的办法求最长不下降子序列。

高度	1	3	6	8	4	5	7
序列长度	1	2	3	4	3	4	5

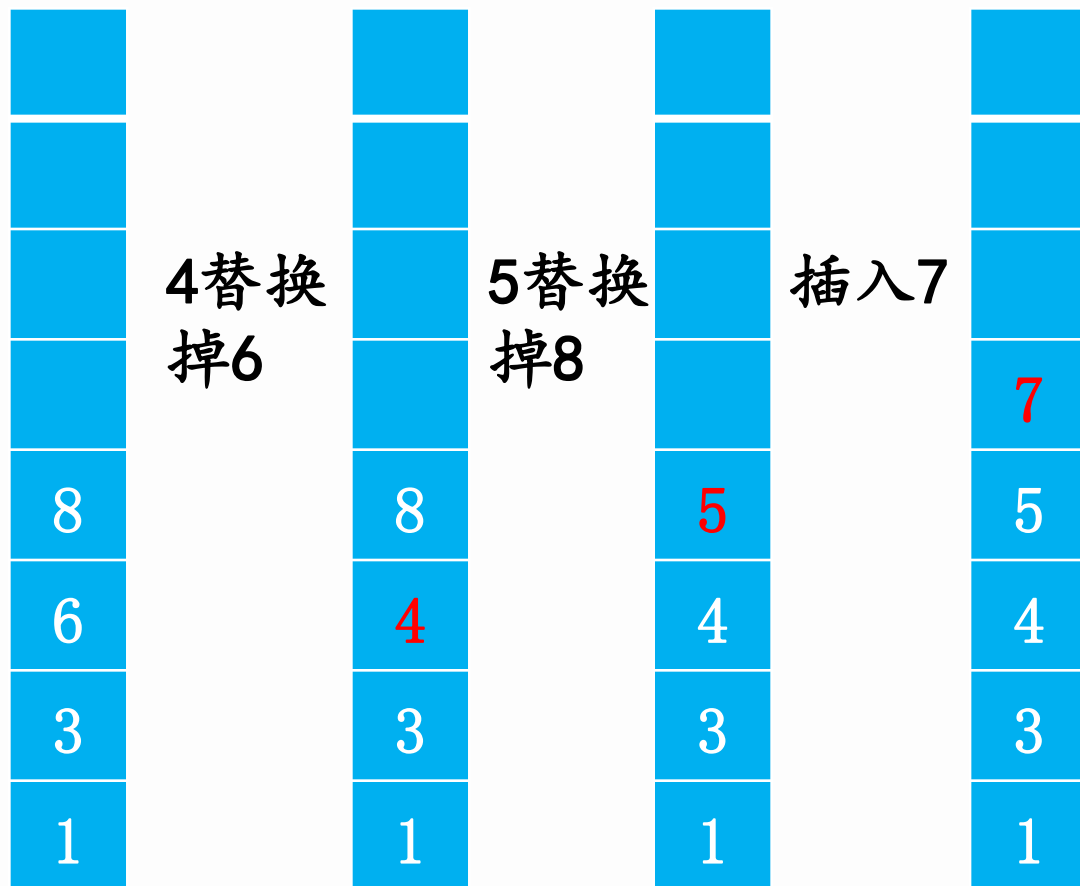


对于每一个高度，寻找他前面高度不高于他的位置，找出其中序列长度最大的，然后+1就是当前最长不下降子序列的长度。对于每一个位置，需要把前面遍历一遍，时间复杂度为 $O(n^2)$ 。

最长不下降子序列

这道题我们可以先用单调栈来做。

对于 $n=7$; 1 3 6 8 4 5 7



最大长度就是5

最长不下降子序列

这样看上去，虽然效率上有一点优化，但是并没有太大的提升，时间复杂度仍然还是 $O(n^2)$ ；但是既然这是个单调栈，那就说明这个栈具有单调性，而我们要在具有单调性的序列中去搜索一个数，很明显可以用二分查找来优化效率，所以总的时间复杂度可以变为 $O(n \log n)$ 。

```
if(h[i]>=s[top])
```

```
{
```

入栈；

```
}
```

```
else
```

```
{
```

二分查找更新的位置

//最后一个比他小的数后面

```
}
```

因为是不下降子序列，所以栈内可能有重复元素，所以二分查找的时候要注意。

更新的意思是，相同长度的序列，让该位置的值最小，这样便于后面的插入。

最大数

题目描述

现在请求你维护一个数列，要求提供以下两种操作：

1. 查询操作 Q L：查询当前数列中末尾L个数中的最大的数，并输出这个数的值。保证 $0 < L \leq \text{len}$
 2. 插入操作 A X：将X对一个固定的常数D取模，将所得答案插入到数列的末尾。 $X \leq \text{int}$ ， $D \leq \text{int}$ 。
- 初始时数列为空。

最大数

该题如何用高级数据结构来做当然是一道裸题，但是如果要用基本的数据结构来做就要困难点，而且由于查询的 L 是定值，所以也不能直接用单点队列来做。

如果用单调栈来做，首先我们需要维护一个单调递减的栈，如果是维护一个单调递增栈，那么前面的很多答案就丢失了。

维护单调递增栈虽然我们把有用的信息都保存在栈里面了，但是我们要如何快速去找到长度为 L 的最大值呢？既然是单调栈，那么肯定可以二分查找栈中第一个位置大于等于查询位置的数，而这个数就是最大值。

其实该题还有更快的办法——并查集，对于每一个出栈的元素，我们记让他出栈的元素作为他的父亲结点，而最后查询他的祖先就可以了。

最大数

```
if(opt=='A')//增加操作
{
    a[++k]=x%p;
    f[k]=k;
    while(top&& a[st[top]]<a[k])//维护单调递减栈
    {
        f[st[top]]=k;
        top--;
    }
    top++;
    st[top]=k;
}
else if(opt=='Q')//查询操作
{
    int tmp=k-x+1;
    printf("%d\n",a[find(tmp)]);
}
```




队列

队列也是NOIP常考内容，主要包括五个方面：

(1) 普通队列模拟

(2) bfs搜索

(3) 最短路

(4) 双端队列

(5) 单调队列优化

(6) 优先队列模拟



逛画展

博览馆正在展出由世上最佳的 M 位画家所画的画, wangjy 想到博览馆去看这几位大师的作品。可是, 那里的博览馆有一个很奇怪的规定, 就是在购买门票时必须说明两个数字, a 和 b , 代表他要看展览中的第 a 幅至第 b 幅画(包含 a 和 b) 之间的所有图画, 而门票的价钱就是一张图画一元。为了看到更多名师的画, wangjy 希望入场后可以看到所有名师的图画(至少各一张)。可是他又想节省金钱。。。

作为 wangjy 的朋友, 他请你写一个程序决定他购买门票时的 a 值和 b 值。

输入格式:

第一行是 N 和 M , 分别代表博览馆内的图画总数及这些图画是由多少位名师的画所绘画的。

其后的一行包含 N 个数字, 它们都介于 1 和 M 之间, 代表该位名师的编号。

解题思路

经过分析我们发现，我们是要找出该序列中包含所有画家至少一副画的一个最短的序列。

我们可以先考虑如何找出刚好包含所有画家的画的序列，比如我们此时在第 i 副画前，往后找到刚好包含所有画家的画的位置。

我们可以边移动边标记该幅画出现了多少次，到了第 i 副画的时候，如果该幅画没有被标记过，我们就标记为1，否则就+1。那我们该如何判断是否所有画都出现了呢？我们可以没新标记一次就用 $\text{cnt}++$ 记录，当 $\text{cnt}==m$ 的时候就表示都出现过了。这样我们每次改变起点都可以找到一个当前最短序列， n 次移动就可以找到最短的了。

解题思路

如果我们使用前面这种方法会特别慢 ($n*n$)，重复特别多，肯定会超时，我们需要避免重复。

假设我们此时找到了第一个区间 $[i, j]$ ，里面包括所有的画家的画，我们可以左端点右移一个，即去掉一幅画，可能会出现两种情况：第一种，画的种数没变 (该画不止一副)，第二种，画的种数变少了 (该画只有一副)，对于第一种情况，我们可以继续右移缩短区间，对于第二种情况，我们要保证画的完整性，就需要在 j 之后找到我们缺少的这幅画，找到之后区间又是完整的，然后已知循环就可以 $O(n)$ 的找完整个区间。每次找到完整的区间的时候，判断区间长度是否为最短，并且记录左右端点。

单调队列

单调队列原理和单调栈相同，唯一不一样的地方在于单调栈中的元素在出栈之前都会对后面起作用，但是单调队列主要适用于前面的元素只在部分时间内起作用，时间一过都不再起作用了。即单调队列主要用于区间问题

单调队列操作和普通队列不太一样，可以在队首删除，可以在队尾插入和删除，所以最好用手写队列，不用STL队列。



上午练习题

MSOJ	1554	蚯蚓
MSOJ	1522	数列操作一
MSOJ	1523	数列操作二
MSOJ	1524	最大数
MSOJ	1184	最长不下降子序列
MSOJ	1133	区间最大值



H小姐

题目描述

H小姐是学校里名副其实的女神，每天想给H小姐献殷勤的追求者总会从女生宿舍排队排到校门口。作为H小姐最要好的朋友，小Z总是会穿梭在队伍中为H小姐物色最佳男友。小Z知道H小姐很看重身高，于是这天她研究起了这群追求者的身高来。

已知共有 N 位追求者排成一列，编号为 $1 \dots N$ ，编号为 i 的追求者身高为 $h[i]$ 。小Z从队伍的某处出发，沿着编号递增的方向经过了 M 位追求者，一边走一边打量着这 M 位追求者的身高，默默记下最高身高以及最高身高的变化次数。例如，若 $M=5$ ，经过的 M 位男生的身高分别为 $3, 1, 2, 5, 7$ ，则小Z先记住了身高为 3 的男生，然后忽略了接下来两位更矮的男生，再然后依次遇到并记住了身高更高的两位男生（也即 5 和 7 ），最终记住了 M 位男生中身高最高的那位（也即 7 ），在这个过程中小Z记住的最高身高变化了 3 次。

现在，你知道小Z记录的最高身高变化了多少次，以及最终记住的最高身高是多少吗？

输出共 $N-M+1$ 行，每行两个数字。

第 i 行的两个数字分别表示若小Z从男生 i 走到男生 $i+M-1$ ，其记录的最高身高变化了多少次，以及最终记住的最高身高是多少。

H小姐

该题相当于每次求固定区间长度的变化次数，看上去和滑动窗口差不多，直接用一个单调队列维护即可。

但是实际上我们会发现有一点问题。如果我维护一个单增队列，那么很明显，前面重复的一截的变化次数我没有办法统计。如果维护一个单减序列，就更不对了。因为该题是求变化次数，而不是求最大或者最小值。

因为该题我们要知道最大高度，是否要变化和有效最大高度有关，所以要维护一个单减序列；但是由于我们又要知道变化量，每次变大才能知道变化量，所以我们需要维护一个单增序列；但是这两个序列冲突了，所以我们可以反向，这样维护一个单减序列就可以了。

H小姐

比如：

5 3

3 1 2 5 7

第一次队列中：7 5 2 变化3次

第二次队列中：5 2 1 变化3次

第三次队列中：3 变化1次

5 3

7 5 2 1 3

第一次：3 2 变化2次

第二次：5 变化1次

第三次：7 变化1次

理想的正方形

题目描述

 展开

有一个 $a*b$ 的整数组成的矩阵，现请你从中找出一个 $n*n$ 的正方形区域，使得该区域所有数中的最大值和最小值的差最小。

输入格式

第一行为3个整数，分别表示 a, b, n 的值

第二行至第 $a+1$ 行每行为 b 个非负整数，表示矩阵中相应位置上的数。每行相邻两数之间用一空格分隔。

输出格式

仅一个整数，为 $a*b$ 矩阵中所有“ $n*n$ 正方形区域中的最大整数和最小整数的差值”的最小值。

理想正方形

该题是一道二维单调队列的裸题，我们可以求出这个 $n*n$ 区间的最大值，再减去这个区间的最小值就可以了。

关键是如何快速求这个最大最小呢？如果仅仅是求一行或者求一列，那么很简单，直接单调队列就可以了，但是如果是 $n*n$ 呢？

其实是一样的，我们先对行求出每个 $1*n$ 的矩形的最大最小值，然后再对列求最大最小值，里面存放的初始值就是行的最大最小值，相当于对所有在这一列的 $1*n$ 的矩形的最大最小值，再求一遍列的最大最小值。

优先队列

优先队列都是用STL的堆实现，有大根堆和小根堆两种，需要自己重载小于运算符。值得注意的是默认的优先队列是大根堆，即堆顶元素是最大的，所以如果我们是大根堆，重载小于运算符的时候还是小于，如果是小根堆，重载小于运算符的时候就要变成大于。一定不能错了

优先队列

优先队列是我们常用的第二个STL，当我们需要把一个数插入一个有序表中的正确位置，如果我们用线性结果做是 $O(n)$ ，特别慢，所以如果要想快点就需要用到树形结构，也就是我们的手写堆，但是代码难度大，我们一般都是使用STL中的优先队列，虽然效率低一点，但是由于代码比较短，所以也是一个不错的选择。

```
#include<queue>
```

```
struct node
```

```
{
```

```
    int x;
```

```
    bool friend operator <(node a,node b)
```

```
    {
```

```
        return a.x>b.x;
```

```
    }
```

```
}
```

```
priority_queue<node> q;
```

优先队默认从大到小排序，如果我们要从大到小排序，就要把<运算符重载为>运算符

操作系统

题目描述

展开

写一个程序来模拟操作系统的进程调度。假设该系统只有一个CPU，每一个进程的到达时间，执行时间和运行优先级都是已知的。其中运行优先级用自然数表示，数字越大，则优先级越高。

如果一个进程到达的时候CPU是空闲的，则它会一直占用CPU直到该进程结束。除非在这个过程中，有一个比它优先级高的进程要运行。在这种情况下，这个新的（优先级更高的）进程会占用CPU，而老的只有等待。

如果一个进程到达时，CPU正在处理一个比它优先级高或优先级相同的进程，则这个（新到达的）进程必须等待。

一旦CPU空闲，如果此时有进程在等待，则选择优先级最高的先运行。如果有多个优先级最高的进程，则选择到达时间最早的。



操作系统

该题是一道优先队列的简单题，只是要注意如果当一个任务正在执行的时候，来了一个高级任务，那么我们需要把这个正在完成的任务未完成部分出队，然后再放进去，其他就是普通的优先队列的处理。



中位数

题目描述

给出一个长度为 N 的非负整数序列 A_i ，对于所有 $1 \leq k \leq (N + 1)/2$ ，输出 $A_1, A_3, \dots, A_{2k-1}$ 的中位数。即前 $1, 3, 5, \dots$ 个数的中位数。

输入输出格式

输入格式：

第1行为一个正整数 N ，表示了序列长度。

第2行包含 N 个非负整数 A_i ($A_i \leq 10^9$)。

输出格式：

共 $(N + 1)/2$ 行，第 i 行为 $A_1, A_3, \dots, A_{2k-1}$ 的中位数。

输入输出样例

输入样例#1： [复制](#)

```
7
1 3 5 7 9 11 6
```

输出样例#1： [复制](#)

```
1
3
5
6
```


中位数

可以考虑线段树来求解区间的中位数，但是比较麻烦，不太好找。我们可以考虑优先队列

开两个堆，一个小根堆，一个大根堆，始终保证小根堆的数 $\leq$ 大根堆的数，并且小根堆的数量 = 大根堆 (+1)

那么这样我们就可以发现，小根堆堆顶的数就是中位数，因为如果一共有 $(2n+1)$ 个数。小根堆有 $n+1$ 个数，除去堆顶的元素还有 n 个，大根堆有 n 个数，并且满足小根堆 $\leq$ 大根堆，那么该数就是中位数

每次新插入一个数，只需要维护两个堆的性质不变就可以了，只需要将堆顶元素和该数比较然后出入队就可以了

中位数

```
if(a[i]>q1.top().num)//当前数比队顶元素大，放入小根堆
{
    node2 t;
    t.num=a[i];
    q2.push(t);
    if(q2.size()>q1.size())//保证中位数在大根堆堆顶
    {
        int tmp=q2.top().num;//取出小根堆的堆顶放入大根堆
        q2.pop();
        node1 t2;
        t2.num=tmp;
        q1.push(t2);
    }
}
else//如果小于则放在大根堆
{
    node1 t;
    t.num=a[i];
    q1.push(t);
    if(q1.size()-q2.size()>1)//大根堆数太多了
    {
        int tmp=q1.top().num;
        q1.pop();
        node2 t2;
        t2.num=tmp;
        q2.push(t2);
    }
}
```

奖学金

题目描述

小张学院有 c 名学生，第 i 名学生的成绩为 a_i ，要获得的奖学金金额为 b_i 。
要从这 c 名学生中挑出 n 名学生发奖学金。这个神秘人物爱好奇特，他希望得到奖学金的同学们的成绩的中位数尽可能大，但同时，他们的奖学金总额不能超过 f 。

输入格式

第一行有三个整数，分别表示要挑出的学生人数 n ，学生总人数 c 和奖学金总额的最大值 f 。

第 2 到第 $(c + 1)$ 行，每行两个整数，第 $(i + 1)$ 行的整数依次表示第 i 名学生的成绩 a_i 和如果要给他发奖学金，则需要发的金额数 b_i 。

输出格式

输出一行一个整数表示答案。如果无法满足神秘人的条件，请输出 -1 。

奖学金

题目要求得奖学生成绩的中位数尽可能大，首先我们看下得奖学生成绩的大小对奖金不具有单调性，因为并不是成绩好奖金就多，所以没有办法二分答案，所以我们可以直接枚举谁作为得奖学生的中位数，当然他必须要满足成绩左右两边的学生数量都 $\geq n/2$

接下来是如何快速验证，如果直接算，总时间复杂度为 $n*n$ ，所以我们思考按成绩排序之后以第 i 位同学和第 $i+1$ 位同学作为中位数的总奖金的关系。

$f[i]$ 表示 i 左边 $c/2$ 个学生奖金之和的最小值，如果新加了一个学生，那么就可以用这个学生去替换前面的最大值或者继续原样。 $g[i]$ 表示后面 $c/2$ 和的最小值，最后再找一个满足条件的就可以了。

种树

题目描述

展开

A城市有一个巨大的圆形广场，为了绿化环境和净化空气，市政府决定沿圆形广场外圈种一圈树。

园林部门得到指令后，初步规划出 n 个种树的位置，顺时针编号1到 n 。并且每个位置都有一个美观度 A_i ，如果在这里种树就可以得到这 A_i 的美观度。但由于A城市土壤肥力欠佳，两棵树决不能种在相邻的位置（ i 号位置和 $i+1$ 号位置叫相邻位置。值得注意的是1号和 n 号也算相邻位置！）。

最终市政府给园林部门提供了 m 棵树苗并要求全部种上，请你帮忙设计种树方案使得美观度总和最大。如果无法将 m 棵树苗全部种上，给出无解信息。

种树

如果是直接贪心，由于该位置种了一棵树那么旁边就不能种树了，所以很容易举出反例，比如10 9 2 9，并且该题数据范围很大是，所以也没有办法使用dp。

该题思路还是贪心，只是可以回撤的贪心，当我们选择了一个点的时候，我们再往里面添加一个新的点，这个点为该点的左边+该点的右边-该点，那么如果我们选了这个新添加的点，就相当于没有选前面那个点而选了那个点左右两边的点，相当于撤销操作，是比较巧妙的。

因为涉及到删加点操作，所以我们用双向链表，这样就可以很快的找到该点左右两边的点修改之后是哪一点

种树

比如 9 8 8 2

先把这些点依次放入一个大根堆里面，保证每次出来的都是最大值。

第一次出队的是9，然后给他添加返回机制，即添加一个点 $8+8-9=7$ ，并且让7入队并且让7的左边等于9的左边的左边，让7的右边等于9的右边的右边，因为选9和选7就相当于选了这两个点。然后8出队，已经标记过了，不管。下一个8出队，标记过了，不管，然后7出队，可以选，那么答案就是 $9+7=16$ ，然后2出队，没有可以选的，结束

种树

```
for(int i=1;i<=m;i++)
{
    while(vis[q.top().id])
        q.pop();
    point tmp=q.top();
    q.pop();
    ans+=tmp.val;
    vis[p[tmp.id].l]=vis[p[tmp.id].r]=1;
    p[tmp.id].val=p[p[tmp.id].l].val+p[p[tmp.id].r].val-p[tmp.id].val;
    tmp.val=p[tmp.id].val;
    q.push(tmp);
    del(tmp.id);
}
```




下午练习题

MSOJ 1531 H小姐

洛谷 1168 中位数

洛谷 1093 奖学金

洛谷 1792 种树

